



# About me (Davide Bettio)



<https://github.com/bettio/> | [davide@uninstall.it](mailto:davide@uninstall.it) |  
<https://uninstall.it/>

- From Padova (Venice area), Italy
- Long-time open-source dev (since ~2005 contributed to KDE Plasma and others).
- Fell in love with Elixir in 2017, while creating Astarte Platform.
- Started the AtomVM project in 2017
- I love hiking!

← Me combining hiking and Elixir by implementing the Meshtastic protocol in Elixir.





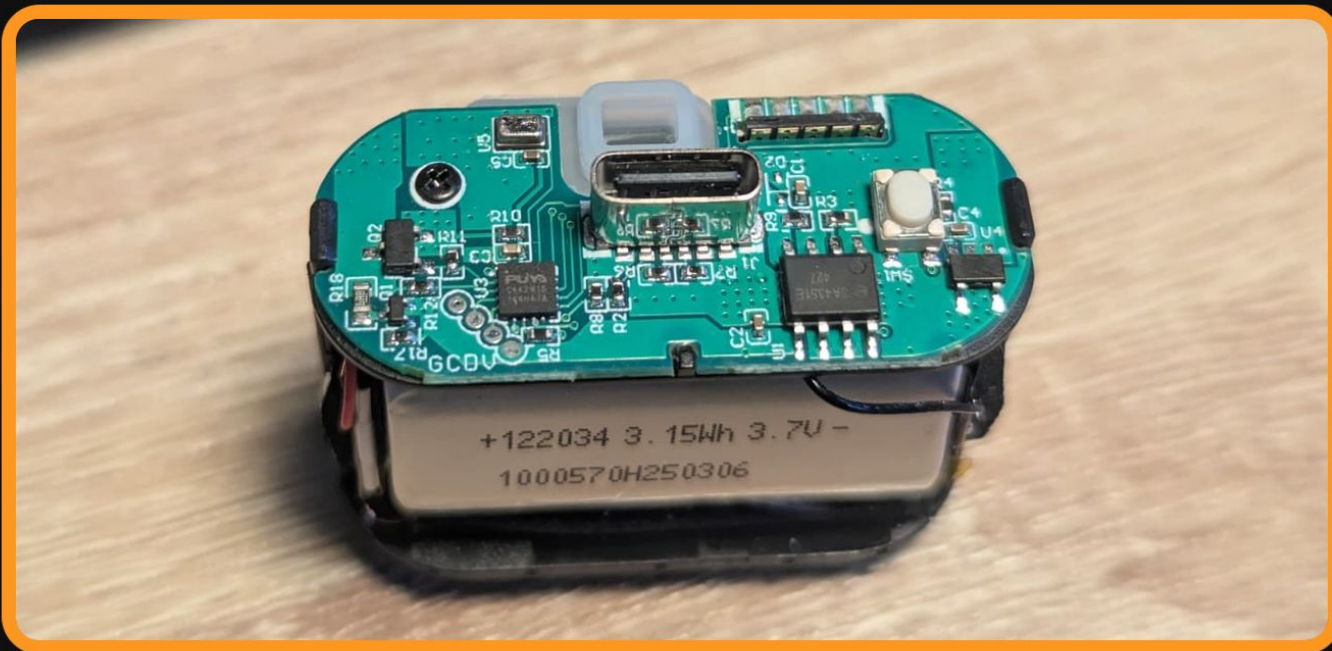
# The State of Computing Today

# Hosting a WebSite on a Disposable Vape

---

2025-09-13 :: Bogdan Ionescu :: 6 min read (1266 words) :: [source](#) :: [report issue](#)

[#programming](#) [#arm](#) [#tools](#) [#electronics](#)



# Is There Really a Computer in There?

**It may not do much<sup>1</sup>, but yes, definitely.**

- ✓ It has a 24 MHz CPU
- ✓ It has 3 kB SRAM memory
- ✓ It has 24 kB of flash storage
- ✓ It can do I/O

<sup>1</sup> To be fair, we went to the Moon with a computer roughly comparable to this.

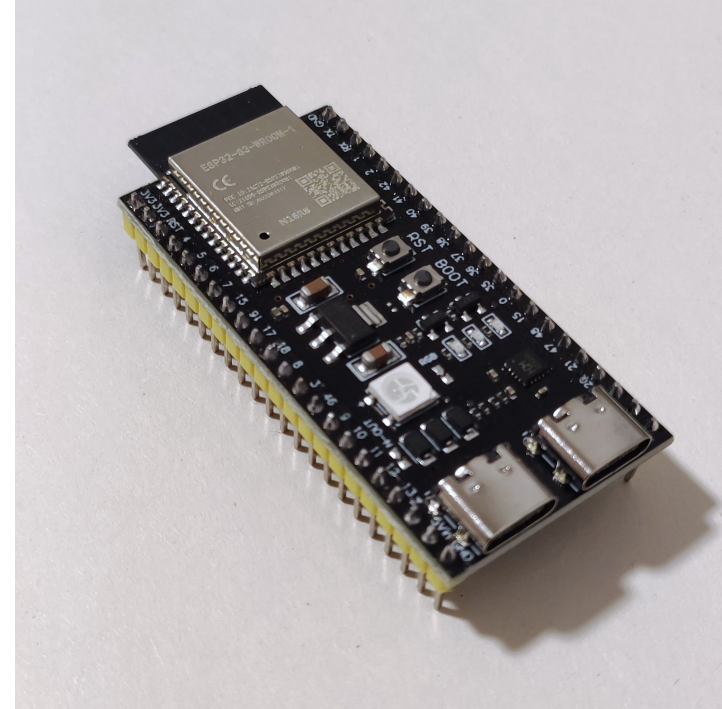




Let's Focus on  
Something  
More Capable

# An ESP32

- ✓ Cost < 5 €
- ✓ Connectivity: WiFi, Bluetooth, etc.
- ✓ Lots of GPIOs & integrated peripherals
- ✓ Low Power / Battery-friendly



## Also an ESP32

- ⚠ Dual Core BUT at 240MHz
- ⚠ RAM: ~500KB - 8MB
- ⚠ Flash: 4MB - 16MB
- ⚠ No advanced operating system

**This is not enough for the BEAM.**

**No BEAM, No Elixir?**



<https://www.reddit.com/r/PallasCats/comments/1d8j3jd/bol/>



# AtomVM to the Rescue

**AtomVM is a lightweight virtual machine designed to run compiled Erlang, Elixir and Gleam code on microcontrollers with limited resources.**

- Key Trade-offs:
  - Memory First: RAM & Flash are precious
  - Portability: New targets in hours, not days
  - Flexible Requirements: Adaptable core

# Other Supported Platforms: Raspberry Pi Pico

- Pico 1/1W (RP2040) → **264 KiB of RAM**, dual core @ 133 MHz+, 2 MiB+ flash
- Pico 2/W (RP2350) → **520 KiB of RAM**, dual core @ 150 MHz, 4 MiB+ flash
- **The 'W' models include Wi-Fi + Bluetooth**
- What's cool? Peripherals & **Programmable I/O (PIO)**

Note: Raspberry Pi Pico ≠ “classic” Raspberry Pi or Raspberry Pi Zero



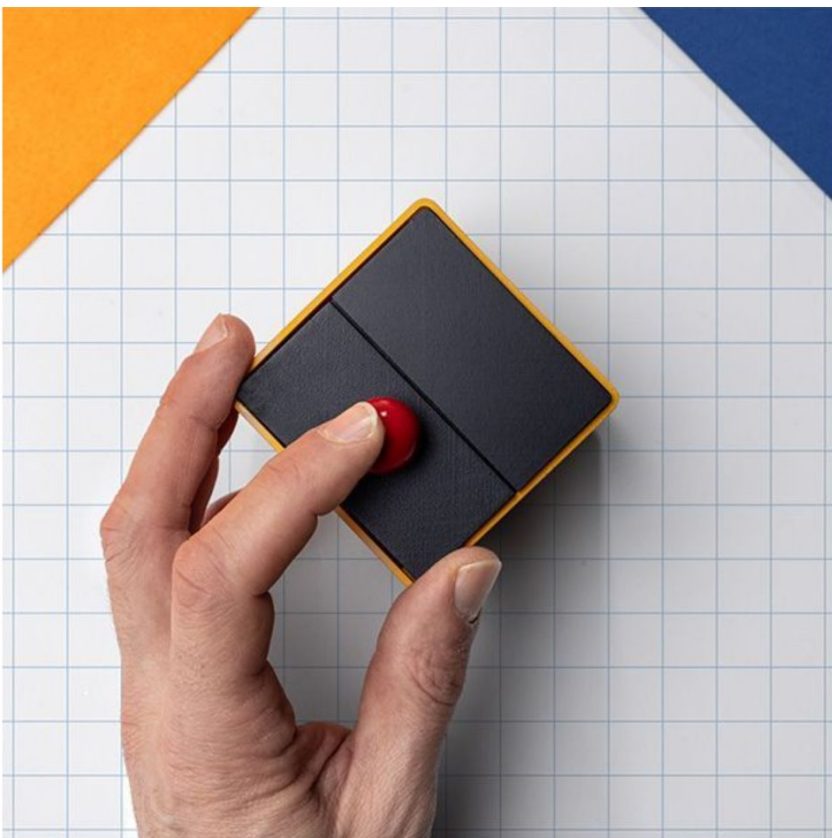
# Supported Platforms: Not Only Microcontrollers

**AtomVM can be used in other unexpected contexts.**

- Linux, FreeBSD, DragonFly BSD, macOS, etc...
- The browser, with Emscripten + WASM

And more are coming, such as WASI, and others.

With AtomVM  
You Can Build Things



## KEY FEATURES

- Over 500 unique sound effects & reactions
- Unlimited choreography combinations - never the same twice
- Fully modular design for easy repairs & customization
- Powered by ESP32 architecture
- Completely open source software - hack it, modify it, make it yours
- Eco-friendly construction from 100% recycled materials
- Exceptional battery life: three months on a single charge



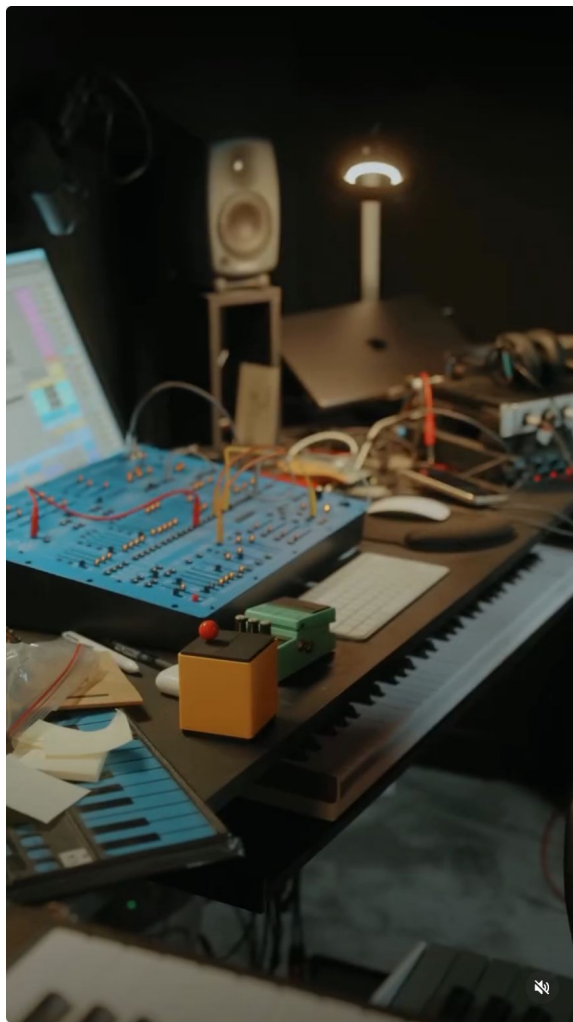
# la machine

by multipli6

The Useless Box : Reloaded



- AtomVM powered
- ESP32-C3
- 32-bit RISC-V single-core @ 160 MHz 400 KiB of SRAM
- 5  $\mu$ A in deep sleep !



# More than **7,000** la machine units running AtomVM have been shipped.

If you have any questions, reach out to Paul Guyot. He developed the software for la machine and has made countless valuable contributions to AtomVM, including JIT support.

← A still from Jean-Michel Jarre's reel, featuring one of the la machine units.

Photo taken from: <https://manulization.com/manuls/manul-16.html>



... and  
of course ...



# The Goatmire 2026 Badge

**AtomVM powered**

You can:

- Exchange contact details via infrared
- Connect it to the internet and display the schedule
- Chat with other participants

And more!





An early 90s PC had  
more RAM  
*but*  
**Constrained  
Doesn't Mean Boring**

Credits: @tomhnl



**How?**



# Big Disclaimer (for Anyone Watching This in 2027)



Source:  
<https://manulization.com/manuls/magellan.html>

- Heads up: AtomVM is still pre-v1.0, which means APIs are not yet stable
- We will break APIs, but the core concepts will remain the same
- The code here might not work forever, but we keep the official documentation and examples up-to-date

See also: <https://doc.atomvm.org/latest/UPDATING.html>

# What you Need / Setup

- Minimal hardware setup: just a USB cable (that's it)
- Your favorite Erlang/Elixir/Gleam setup

## Next Step: exatomvm

- Add `{:exatomvm, github: "AtomVM/exatomvm", runtime: false}` to `mix.exs`
- It provides a number of Mix tasks to build & flash your project
- On ESP32: **`mix atomvm.esp32.install`** (downloads a pre-built binary)

# Configuring mix.exs

Add an `:atomvm` entry to the keyword list returned by `project/0` in `mix.exs`.

```
def project do
  [
    [...],
    atomvm: [
      start: Blink # the module with our start/0 entry point function
    ]
  ]
end
```

# The Physical Computing Hello World

```
defmodule Blink do
  @pin 2

  def start() do
    GPIO.set_pin_mode(@pin, :output)
    loop(:high)
  end

  defp loop(level) do
    GPIO.digital_write(@pin, level)
    Process.sleep(200)
    loop(toggle(level))
  end

  defp toggle(:high), do: :low
  defp toggle(:low), do: :high
end
```



This will not work on the Goatmire 2026 badge!  
The badge uses addressable RGB LEDs instead.

Check how to use them:

[github.com/bettio/atomvm\\_workshop\\_examples](https://github.com/bettio/atomvm_workshop_examples)



# What's a GPIO?

- **GPIO** stands for **G**eneral **P**urpose **I**nput/**O**utput.
- Think of them as simple digital pins that can be either an input or an output
- They can be set to high (e.g., 3.3V) or low (0V / Ground).

For a regular LED, this means it's either fully on or fully off. No fading.

# The AtomVM Workflow

- Add `{:exatomvm, github: "AtomVM/exatomvm", runtime: false}` to `mix.exs` ✓
- Write Elixir (like always!) ✓
- (Behind the scenes: compile, like always!)
- (Behind the scenes: `pack, mix atomvm.packbeam → myapp.avm`)
- **Flash, run one command: (e.g. `mix atomvm.esp32.flash`)**

# Demo



HONESTLY, I WILL NOT DO A BLINKING LED DEMO. TRUST ME, IT WORKS. I WILL NOT EVEN TRY SHOWING A MICROSCOPIC LED TO THE AUDIENCE. IF I BRING BOARDS AND ELECTRONICS ON AN AIRPLANE I MIGHT BE MISIDENTIFIED AS A TERRORIST. DEMOS ALWAYS FAIL ANYWAY.

# Time for `mix atomvm.esp32.monitor`

- After flashing, use the serial monitor to read `I0.puts` and `I0.inspect` output
- Don't run the serial monitor and the flasher at the same time
- It may require some configuration

(When it doesn't work you can try minicom as an alternative)

# Circuits Pro-tips

## Don'ts

1. **Never mess with “GND”** (the ground): if you connect the GND pin to something that is not ground/0V you are likely going to fry your device
2. **Respect polarity:** components like LEDs and some capacitors have positive and negative sides: connecting them backward = 💀
3. **Don't mix voltage levels:** sending 5V into a 3.3V pin can permanently damage the chip unless the pin is explicitly '*5V tolerant*'
4. **Always connect an antenna** before powering on a radio. Without it, the transmitter can be damaged

Do: **Double-check all your connections before powering up your device!**

# Handling a Button Press

Goal: We want to know when a button is pressed

- The naive way is "polling": constantly looping to check the button's GPIO pin
  - The problem? This keeps the CPU busy doing nothing and drains the battery. This is called "busy-waiting"
- A better way: Interrupts
  - A hardware interrupt tells the CPU to pause its current task and handle something important \*right now\*
  - In AtomVM, we translate these hardware interrupts into standard Elixir messages



# Interrupts in Elixir

First, we configure the GPIO as an input and tell it to trigger an interrupt on a 'falling edge' (when the button is pressed):

```
:gpio.set_direction(gpio, gpio_num, :input)
:gpio.set_int(gpio, gpio_num, :falling)
```

The **hardware event is safely delivered to your process's mailbox as a message**. No callbacks, no polling: just the actor model you already know and love. For example, add this to our GenServer:

```
def handle_info({:gpio_interrupt, gpio_num}, state) do
  IO.puts "Button pressed"
  {:noreply, state}
end
```



**Great, I'm blinking an LED and reading a button. Now what?**

Source: <https://www.flickr.com/photos/tambako/31556104335/in/photostream/>

# Peripherals!

- Connecting to the outside world: peripherals!
- Usually, a *bus* lets you connect multiple devices to the same set of GPIO pins
- Many flavors
  - **I2C** → 2 wires, synchronous, slow-to-medium speeds
  - **SPI** → 4-wire bus (or more), synchronous, faster than I<sup>2</sup>C, good for displays
  - **UART** → 2-wire bus, point-to-point connection. TL;DR: a serial port
- Most sensors, displays, and other modules use one of these standard communication protocols (or "buses")

# Native vs Elixir Components

- AtomVM APIs (`spi`, `i2c`, `gpio`) let you build libraries for **complex peripherals in pure Elixir/Erlang (no C required)**.
- High-performance or very low-level hardware access?
  - Use **Native components (NIFs and Ports)**, just like the regular BEAM
  - The catch: using native components requires a custom AtomVM build using the device's SDK (like the ESP-IDF)

# Displaying stuff: AtomGL

- A native component for rendering to a display
- Displays any list of items:

```
{:text, 16, 16, :default16px, 0xFFFFFFFF, 0x404040, title},  
{:image, div(320 - 64, 2), div(240 - 64, 2), 0x404040, error_image}
```

- There is an additional `avm_scene` library that provides some scaffolding for managing the displayed scene using a `gen_server`-style interface:

```
def handle_info(:show_foo, %{width: width, height: height} = state)  
do  
    {:noreply, state, [{:push, items}]}  
end
```

See also: <https://github.com/atomvm/atomgl>

# What about the internet?

- There is a handy network module
- AtomVM has support for `gen_tcp`, `gen_udp` and `socket`
- `http_server` and `ahhttp_client` modules
- `mdns` for finding your device on the network

See also: <https://doc.atomvm.org/latest/network-programming-guide.html>

# Setting up Wi-Fi & Handling HTTP Requests

```
def connect_wifi(ssid, psk) do
  config = [
    ssid: ssid,
    psk: psk,
    dhcp_hostname: "atomvm-badge"
  ]

  case :network.wait_for_sta(config, 15_000) do
    {:ok, {ip, _netmask, _gateway}} ->
      IO.inspect(ip, label: "WiFi connected")
      :ok

    {:error, reason} ->
      IO.inspect(reason, label: "WiFi failed")
      {:error, reason}
  end
end
```

```
router = [
  {~c"★", __MODULE__, []}
]

:http_server.start_server(8080, router)

[...]

def handle_req(~c"GET", [], conn) do
  body =
    "<html>\n" <> [...]

  :http_server.reply(200, body, conn)
end
```



# AtomVM has some additions/differences

Compared to the regular BEAM, AtomVM has some extensions to the standard BEAM (usually prefixed with atomvm or avm), e.g.:

- `:atomvm.read_priv/2` → reads a binary file stored in a loaded .avm file
- `:atomvm.posix_*` → posix functions, they mimic unistd.h ones

**.beam files are packed into .avm files** which are designed to be written directly to flash memory (no filesystem needed).

(BTW: they all start with `#!/usr/bin/env AtomVM` →

next idea: making startup-time-free CLI tools with AtomVM ;) )

# Talking About Differences



In order to be lightweight we had to remove weight, so we didn't implement some features and modules you might expect on the BEAM, such as telco protocols, legacy functions or some modules like digraph, gb\_trees, etc...

# We Implemented What We Think Matters

**AtomVM has an opinionated subset of features that provides what we believe makes the BEAM ecosystem so cool.**

AtomVM focuses on what we love about the BEAM ecosystem. You still get supervision trees, `gen_server`, `gen_statem`, familiar data types, and Erlang distribution.

**Also the VM itself is powerful enough to be able to run Erlang and Elixir compilers.**

# When an Erlang Module Is Missing.

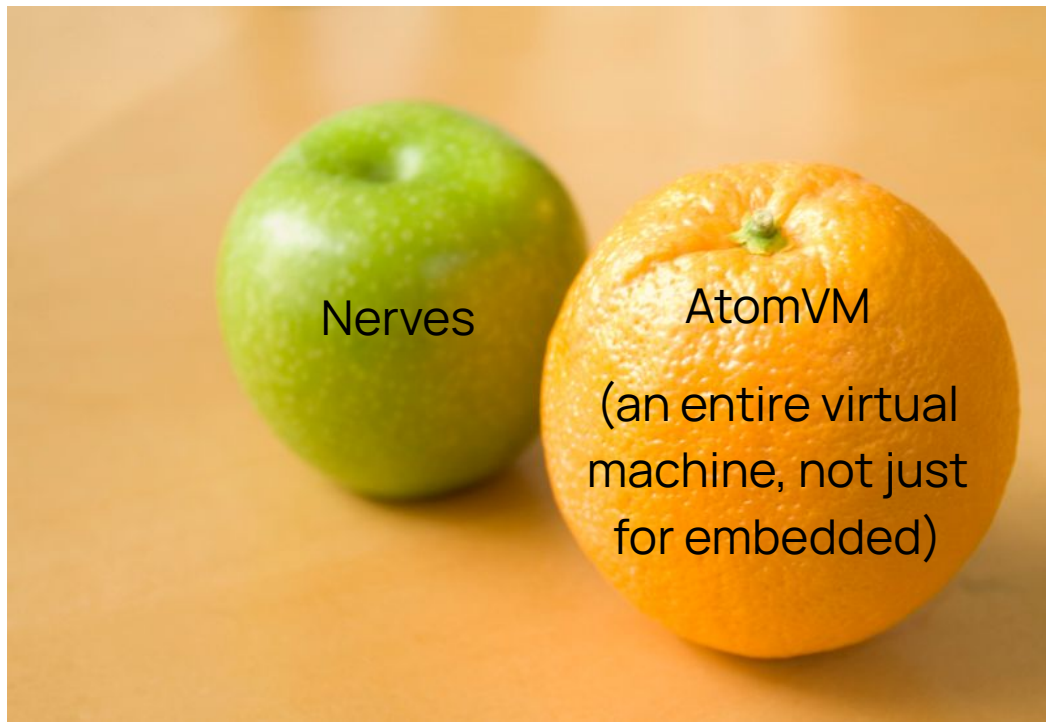
You can likely safely import missing modules with no changes from Erlang/OTP and they will work.





# The Future

# Comparison with Nerves



[https://en.wikipedia.org/wiki/Apples\\_and\\_oranges#/media/File:Apple\\_and\\_Orange\\_-\\_they\\_do\\_not\\_compare.jpg](https://en.wikipedia.org/wiki/Apples_and_oranges#/media/File:Apple_and_Orange_-_they_do_not_compare.jpg)

Let's compare apples and oranges anyway.

AtomVM on MCUs, TL;DR:

- Hello world footprint: 512 KiB of flash, 32 KiB of RAM
- Perfect for cheap low power devices
- ...

# Nerves & AtomVM

Right now AtomVM and Nerves have a very different set of APIs

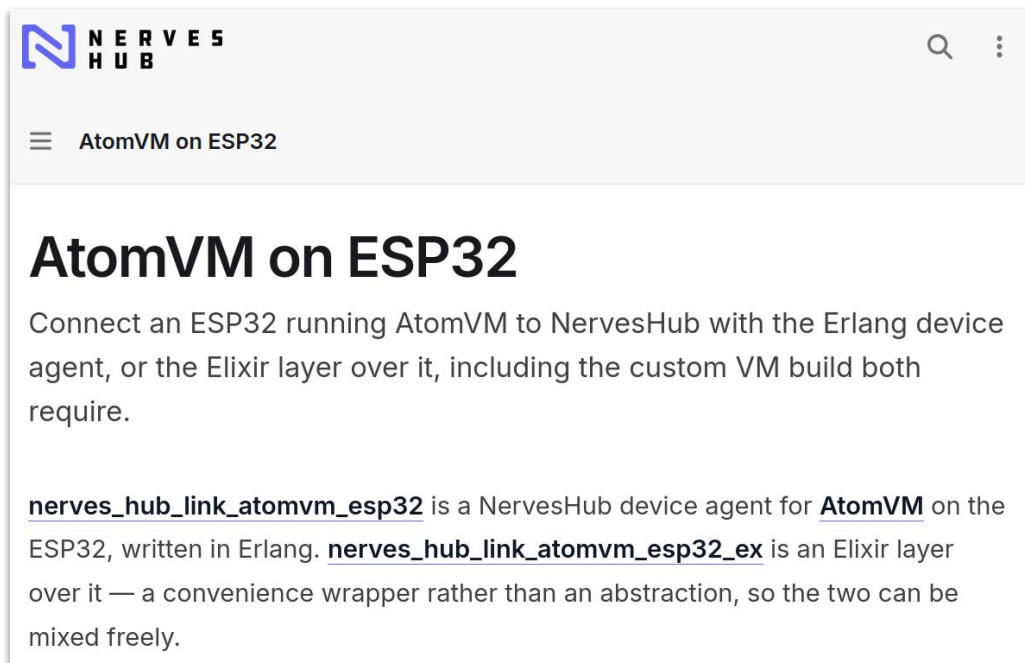
But **in the future I believe AtomVM Elixir users will use a similar (or maybe the same) set of APIs** provided by Nerves, such as `circuits_gpio` and so on...





# Nerves Hub

AtomVM and Nerves Ecosystem integration has already started with Nerves Hub.



The screenshot shows the Nerves Hub website interface. At the top left is the Nerves Hub logo, consisting of a stylized 'N' and the text 'NERVES HUB'. To the right of the logo is a search icon and a menu icon. Below the logo, there is a navigation bar with a hamburger menu icon and the text 'AtomVM on ESP32'. The main content area has a large heading 'AtomVM on ESP32'. Below the heading is a paragraph: 'Connect an ESP32 running AtomVM to NervesHub with the Erlang device agent, or the Elixir layer over it, including the custom VM build both require.' Below this paragraph is another paragraph: '[nerves\\_hub\\_link\\_atomvm\\_esp32](#) is a NervesHub device agent for **AtomVM** on the ESP32, written in Erlang. [nerves\\_hub\\_link\\_atomvm\\_esp32\\_ex](#) is an Elixir layer over it — a convenience wrapper rather than an abstraction, so the two can be mixed freely.'

# Releases

- **Soon to be released: v0.7.0:** <https://github.com/atomvm/AtomVM/releases>
  - Up to OTP-30
  - Erlang Distribution, Big Integers, JIT / AoT, etc...
  - Well tested, focused on stability
- **Development branch: main**
  - <https://github.com/atomvm/AtomVM/tree/main>
  - Moving target, still pretty high quality
  - Use `main` only if you need unreleased features and are comfortable with breaking changes.

# What's Next

Soon:

- Bitstrings (as in not byte aligned binaries)
- Native Records

Future:

- More devices & peripherals (Zephyr devices, Bluetooth, Zigbee/Thread, etc...)
- Even better tooling & DevX
- Stable APIs (path to 1.0)

TL;DR

AtomVM APIs are not yet perfect  
and Developer Experience will be improved,  
but generally speaking **AtomVM is ready for prime  
time.**

# Support this Project

Contributions of all kinds are welcome.

I'm also looking for sponsors to help make AtomVM sustainable in the long term.

❤️ <https://github.com/sponsors/atomvm> ❤️





# Join Us

<https://atomvm.org/>

Discord: <https://discord.gg/QA7fNjm9Nw>

Telegram: <https://t.me/atomvm>

Documentation: <https://doc.atomvm.org/>